

EV2 CLANDESTINOZ

MANUAL DE IMPLEMENTACIÓN Y DESARROLLO

Sistema de gestión de nightclub: Backend · Web · iOS · Android · Integración POS

Guía paso a paso para llevar el sistema desde el prototipo actual hasta un servidor con datos reales. Cada paso indica qué hacer, cómo hacerlo, cuánto tiempo toma y cómo verificar que quedó bien.

Versión	1.0
Fecha	3 de septiembre de 2026
Duración total estimada	13 semanas (1 desarrollador de tiempo completo)
Servidor objetivo	VPS Linux (Ubuntu 24.04) con Docker
POS a integrar	SoftRestaurant11 (con credenciales disponibles)
Asignado a	
Fecha de inicio	

Índice

0	Cómo usar este manual
1	Estado real del sistema (lo que hay y lo que falta)
2	Arquitectura objetivo
3	Calendario general y horas por fase
4	FASE 0 — Orden y base de trabajo (Semana 1)
5	FASE 1 — Base de datos unificada y núcleo de la API (Semanas 2–3)
6	FASE 2 — Persistencia de los módulos de negocio (Semanas 3–5)
7	FASE 3 — Tiempo real y pagos (Semanas 5–7)
8	FASE 4 — Integración real con SoftRestaurant11 (Semanas 6–8)
9	FASE 5 — Conectar las apps web a la API (Semanas 7–9)
10	FASE 6 — Apps nativas iOS y Android (Semanas 8–12)
11	FASE 7 — Servidor, staging y datos reales (Semanas 9–11)
12	FASE 8 — Seguridad, cumplimiento y cierre (Semanas 11–13)
13	Riesgos y qué hacer si ocurren
14	Definición de "listo para producción"
15	Hoja de control de avance (para firmar)

0. Cómo usar este manual

Para quién es: el desarrollador o equipo que va a ejecutar el trabajo. Se asume conocimiento de Node.js, SQL, Docker y Linux básico. No se asume conocimiento previo del proyecto.

Cómo está organizado: el trabajo se divide en 9 fases (0 a 8). Cada fase tiene pasos numerados (por ejemplo, **Paso 1.3** = fase 1, paso 3). Cada paso muestra:

- **Qué hacer** — la tarea concreta.
- **Cómo** — instrucciones, comandos o archivos a tocar.
- **Tiempo** — estimado en horas de trabajo efectivo (etiqueta en el encabezado del paso).
- **Verificar** — cómo comprobar que el paso quedó bien. No se avanza al siguiente paso si la verificación falla.

Reglas de trabajo: (1) Todo cambio se hace en una rama de git y se integra por pull request. (2) Nunca se prueba contra la caja activa del club ni con dinero real hasta la Fase 7. (3) Al terminar cada fase se llena la hoja de control de la sección 15. (4) Si un paso toma más del doble del tiempo estimado, se detiene y se reporta.

Convención de tiempos: 1 día = 8 horas efectivas. 1 semana = 5 días = 40 horas.

1. Estado real del sistema

La carpeta `EV2-app` contiene documentación que describe el sistema como "100% listo para producción". **Eso no es correcto.** Antes de trabajar es indispensable conocer el estado real, porque muchas decisiones de este manual salen de aquí.

1.1 Lo que existe y sirve de base

Componente	Archivo(s)	Estado real
API principal	<code>server/nightclub-api.js</code>	Express + Postgres + Redis + WebSocket. Auth JWT, bebidas, inventario, pedidos, mesas, dashboard, webhook POS. 17 endpoints (la documentación dice 80).
Servidor WebSocket	<code>server/server.js</code>	Funciona, pero guarda 5 mesas fijas en memoria . Al reiniciar se pierde todo.
API de reservaciones	<code>server/reservations-api.js</code>	Router completo (8 rutas) con Stripe. Nunca se monta en la API principal y <code>stripe</code> no está en <code>package.json</code> .
Esquema de BD	<code>init-nightclub.sql</code> , <code>init-reservations.sql</code> , <code>init.sql</code>	Tres esquemas parcialmente duplicados. Solo el primero se carga en Docker.
Módulos de negocio	<code>dance-floor-manager.js</code> , <code>flirt-system.js</code> , <code>employee-account-system.js</code> , <code>pos-integration-system.js</code> , <code>pricing-system.js</code> , <code>taxi-system.js</code> , <code>tip-dancer-system.js</code> , <code>valet-parking-system.js</code> , <code>nightclub-tables.js</code>	Lógica útil, pero 100% en memoria (cero consultas a BD) y sin rutas HTTP . Son clases sueltas.
Apps web	<code>index-ev2-branded.html</code> , <code>table-selector.html</code> , <code>tip-dancers-system.html</code> , <code>employee-portal.html</code> , <code>safe-departure.html</code> , <code>pricing-dance-floor-flirt.html</code>	Interfaces completas con marca, pero ninguna llama a la API . Todo son datos simulados.
iOS	11 archivos <code>.swift</code>	Código suelto. Sin proyecto Xcode , sin assets, sin Info.plist.
Android	<code>EV2Android.kt</code> , <code>ReservationSystem.kt</code>	Código suelto. Sin proyecto Gradle , sin manifest.

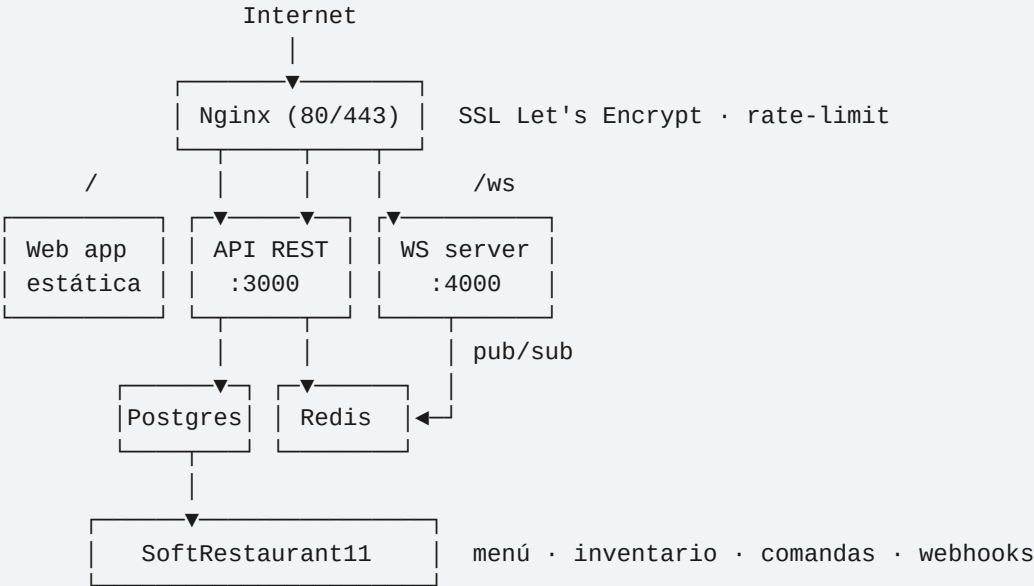
Infraestructura	<code>docker-compose.yml</code> , 3 Dockerfiles, <code>nginx-nightclub.conf</code> , <code>.env.example</code>	Estructura correcta. Errores: el Dockerfile web sirve <code>index.html</code> en vez de <code>index-ev2-branded.html</code> ; <code>Dockerfile.nightclub</code> copia solo un archivo; <code>package.json</code> apunta a <code>connect-server.js</code> que no existe.
-----------------	--	---

1.2 Defectos que bloquean cualquier prueba real

#	Defecto	Se corrige en
1	<code>redis.createClient({host, port})</code> es sintaxis de redis v3; el proyecto usa v4. Redis nunca conecta.	Paso 0.5
2	Reservaciones y Stripe no integrados ni instalados.	Pasos 0.4 y 1.4
3	Tres SQL con tablas duplicadas (<code>users</code> , <code>inventory</code> , <code>drink_orders</code>).	Paso 1.1
4	No hay control de versiones (no existe <code>.git</code>).	Paso 0.1
5	Sin <code>package-lock.json</code> , sin tests, sin linter.	Pasos 0.4 y 1.6
6	<code>.env</code> con valores de desarrollo está en la carpeta; riesgo de subirlo a git o al servidor.	Paso 0.1
7	Propinas, retiros y pagos viven en memoria: dinero que se pierde al reiniciar.	Fase 2
8	Ninguna pantalla web está conectada al backend.	Fase 5

Conclusión: se tiene un prototipo visual y arquitectónico muy avanzado y un backend base real. Lo que falta es trabajo de **integración y persistencia**: unir piezas, guardar en base de datos, conectar pantallas y desplegar. Eso es lo que este manual organiza.

2. Arquitectura objetivo



Apps iOS / Android → API REST + WS (mismo contrato OpenAPI que la web)

2.1 Decisiones fijas (no se cambian sin aprobación)

Tema	Decisión
API	Una sola API Express que monta routers por módulo: <code>/api/auth</code> , <code>/api/nightclubs</code> , <code>/api/reservations</code> , <code>/api/employees</code> , <code>/api/tips</code> , <code>/api/taxi</code> , <code>/api/valet</code> , <code>/api/pricing</code> , <code>/api/pos</code> .
Base de datos	PostgreSQL 15, un solo esquema, migraciones numeradas en <code>server/migrations/</code> .
Tiempo real	El WS server no guarda estado propio. La API publica eventos en Redis pub/sub y el WS los reenvía a clientes autenticados por JWT.
Web	HTML + Tailwind sin build. Módulo compartido <code>web/js/api.js</code> para token, fetch y WS.
Móvil	Proyectos reales Xcode (SwiftUI, iOS 16+) y Android Studio (Compose, minSdk 26). Cliente generado desde <code>openapi.yaml</code> .
Pagos	Stripe (tarjeta, Apple Pay, Google Pay, USD). Mercado Pago (México: tarjeta, OXXO, SPEI). Zelle / Cash App / depósito = "pago manual verificado por manager".
Retiros de empleados	Primera versión: registro + aprobación manual. Pago automático a CLABE/ACH queda para después.
Servidor	VPS Ubuntu 24.04 con Docker Compose. Producción y staging en el mismo VPS al inicio, separados por subdominio y stack.

2.2 Estructura de carpetas objetivo

```
EV2-app/
├── server/
│   ├── src/
│   │   ├── index.js          # arranque de la API
│   │   ├── ws.js             # servidor WebSocket
│   │   ├── routes/           # un archivo por módulo
│   │   ├── services/         # lógica de negocio (módulos actuales refactorizados)
│   │   ├── db/               # pool, repositorios
│   │   └── middleware/       # auth, validación, errores, rate-limit
│   ├── migrations/           # 001_init.sql, 002_reservations.sql, ...
│   ├── seeds/                # datos de desarrollo
│   ├── tests/
│   ├── openapi.yaml
│   └── package.json
├── web/                      # HTML de la app + js/api.js + images/
├── mobile/ios/               # proyecto Xcode
├── mobile/android/           # proyecto Gradle
├── deploy/                   # docker-compose.yml, docker-compose.prod.yml, nginx/, scripts/
├── docs/                     # toda la documentación actual (.md / .txt / .html de guías)
├── .github/workflows/        # CI/CD
├── .gitignore
├── .env.example
└── README.md
```

3. Calendario general y horas por fase

Fase	Nombre	Semanas	Horas	Hito al terminar
0	Orden y base de trabajo	1	40	Repositorio en git; <code>docker compose up</code> sano
1	BD unificada y núcleo de la API	2–3	80	H1: API sobre Postgres real, tests en verde, OpenAPI
2	Persistencia de módulos de negocio	3–5	100	Ningún módulo en memoria
3	Tiempo real y pagos	5–7	90	H2: pago sandbox extremo a extremo + eventos en <1 s
4	Integración SoftRestaurant11	6–8	70	H3: pedido de la app aparece en el POS de prueba
5	Conectar apps web	7–9	90	H4: flujo cliente y staff con datos del servidor
6	Apps nativas iOS y Android	8–12	150	H6: TestFlight / prueba interna
7	Servidor, staging y datos reales	9–11	80	H5: piloto 1 en el club
8	Seguridad, cumplimiento y cierre	11–13	60	H7: producción
Total			~760 h	≈ 13 semanas con 1 persona · ≈ 7–8 con 2 personas

Las fases 2 a 5 se solapan porque comparten trabajo. Si hay dos personas: la persona A lleva backend (fases 1–4) y la persona B lleva web y móvil (fases 5–6) desde la semana 3, usando el contrato OpenAPI como acuerdo.

3.1 Diagrama de semanas

Fase \ Semana	1	2	3	4	5	6	7	8	9	10	11	12	13
F0 Base													
F1 BD + API													
F2 Módulos													
F3 RT + Pagos													
F4 POS													
F5 Web													
F6 Móvil													
F7 Servidor													
F8 Seguridad													
Pilotos / Prod.													

4. FASE 0 — Orden y base de trabajo

Semana 1 · 40 horas. Objetivo: dejar el proyecto bajo control de versiones, ordenado, y con `docker compose up` funcionando en una máquina limpia. **No se escribe funcionalidad nueva en esta fase.**

Paso 0.1 — Control de versiones y protección de secretos

3 h

Qué hacer: Inicializar git, excluir archivos sensibles y subir a un repositorio privado.

Cómo:

```
cd EV2-app
git init
cat > .gitignore <<'EOF'
node_modules/
.env
.env.*
!.env.example
ssl/
Thumbs.db
*.log
dist/
build/
.DS_Store
EOF
git add .
git commit -m "Estado inicial del prototipo EV2"
git remote add origin <URL del repositorio privado>
git push -u origin main
```

- Crear el repositorio privado en GitHub o GitLab antes de hacer push.
- Abrir `.env`, copiar cualquier valor real a un gestor de contraseñas y dejar en el archivo solo valores de desarrollo.

Verificar: `git status` limpio; `git ls-files | grep .env` devuelve solo `.env.example`; el repositorio se ve en el remoto.

Paso 0.2 — Reorganizar carpetas

4 h

Qué hacer: Mover archivos a la estructura de la sección 2.2, sin cambiar aún el contenido del código.

Cómo:

```
git checkout -b fase0/reorganizar
mkdir -p docs web/images server/src/{routes,services,db,middleware} server/migrations
server/seeds server/tests deploy mobile/ios mobile/android
git mv *.md *.txt docs/ # documentación
git mv APP_RUNNING_NOW.html QUICK_TESTING_GUIDE.html SYSTEM_INDEX.html app-preview.html
development-timeline.html launch-dashboard.html docs/
git mv index-ev2-branded.html index.html table-selector.html tip-dancers-system.html
employee-portal.html safe-departure.html pricing-dance-floor-flirt.html web/
git mv images/* web/images/
git mv *.swift mobile/ios/
git mv *.kt mobile/android/
git mv docker-compose.yml Dockerfile nginx.conf nginx-nightclub.conf .dockerignore deploy/
git mv server/*.sql server/migrations/
git mv server/nightclub-api.js server/src/index.js
git mv server/server.js server/src/ws.js
git mv server/reservations-api.js server/src/routes/reservations.js
git mv server/{dance-floor-manager,flirt-system,employee-account-system,pos-integration-
system,pricing-system,taxi-system,tip-dancer-system,valet-parking-system,nightclub-
tables}.js server/src/services/
git mv server/restaurant-api.js docs/legacy-restaurant-api.js # versión antigua, solo
referencia
```

- Escribir un `README.md` nuevo en la raíz (media página): qué es el proyecto, cómo arrancarlo, dónde está cada cosa y una nota clara de que la documentación en `docs/` describe la visión, no el estado.

Verificar: la raíz tiene solo carpetas + `README.md`, `.gitignore`, `.env.example`. Commit y PR.

Paso 0.3 — Documentar las decisiones (README + ADR corto)

2 h

Qué hacer: Crear `docs/DECISIONES.md` con la tabla de la sección 2.1 para que cualquier persona nueva sepa qué está decidido.

Verificar: el archivo existe y está enlazado desde `README.md`.

Paso 0.4 — Corregir package.json e instalar dependencias**3 h**

Qué hacer: Dejar scripts correctos, agregar las librerías que faltan y generar el lockfile.

Cómo:

```
cd server
npm pkg set scripts.start="node src/index.js" scripts.dev="nodemon src/index.js" \
  scripts.ws="node src/ws.js" scripts.migrate="node src/db/migrate.js" \
  scripts.seed="node seeds/run.js" scripts.test="jest --runInBand" scripts.lint="eslint
src"
npm install stripe mercadopago dotenv helmet express-rate-limit zod pino pino-http
npm install -D jest supertest eslint nodemon
npm install # genera package-lock.json
```

Verificar: `package-lock.json` existe y está en git; `npm ls` no muestra errores.

Paso 0.5 — Corregir el cliente Redis (v4)**1 h**

Qué hacer: En `server/src/index.js` reemplazar la creación del cliente y conectar explícitamente.

```
const redisClient = redis.createClient({
  socket: { host: process.env.REDIS_HOST || 'localhost', port:
Number(process.env.REDIS_PORT || 6379) }
});
redisClient.on('error', (err) => logger.error({ err }, 'Redis error'));
await redisClient.connect(); // dentro de una función async de arranque
```

Verificar: al arrancar, el log muestra conexión a Redis sin errores y `redis-cli ping` responde `PONG`.

Paso 0.6 — Corregir Dockerfiles y docker-compose**4 h**

Qué hacer: Que las imágenes incluyan todo el código y sirvan la web correcta.

- `deploy/Dockerfile.api`: copiar `package.json`, `package-lock.json`, `src/`, `migrations/` completos; `CMD ["node","src/index.js"]`.
- `deploy/Dockerfile.ws`: igual, con `CMD ["node","src/ws.js"]`.
- `deploy/Dockerfile.web`: usar `nginx:alpine` y copiar la carpeta `web/` completa a `/usr/share/nginx/html`; `index-ev2-branded.html` se renombra a `index.html` y el `index.html` anterior pasa a `docs/legacy-index.html`.
- En `docker-compose.yml` ajustar rutas de `build.context` y `dockerfile`, y montar `../server/migrations` ya no como `initdb` (las migraciones se aplican con `npm run migrate` desde la Fase 1).

Verificar: `docker compose -f deploy/docker-compose.yml build` termina sin error.

Paso 0.7 — Arranque local completo**3 h****Qué hacer:** Levantar todo y probar salud.

```
cp .env.example .env
docker compose -f deploy/docker-compose.yml up -d
docker compose -f deploy/docker-compose.yml ps
curl http://localhost:3000/health
curl http://localhost:4000/health
curl -I http://localhost:8080/
```

Verificar: los 6 contenedores en estado `healthy`; los tres `curl` responden 200. Abrir `http://localhost:8080` en el teléfono (misma red Wi-Fi) y ver la pantalla con marca EV2.**Paso 0.8 — CI mínimo****3 h****Qué hacer:** Crear `.github/workflows/ci.yml` que en cada PR ejecute `npm ci`, `npm run lint` y `npm test` (aunque todavía no haya tests) y construya las imágenes Docker.**Verificar:** el PR muestra la marca verde de CI.**Paso 0.9 — Confirmar documentación oficial de SoftRestaurant11****4 h****Qué hacer:** Con las credenciales disponibles, obtener del proveedor (Nacional Software) la documentación real del mecanismo de integración: API REST, conector a SQL Server, o exportación de archivos. Anotar en `docs/POS_REAL.md`: URL base, método de autenticación, endpoints reales para mesas, menú, inventario y órdenes, formato de webhooks (si existen) y si hay ambiente de pruebas.

Importante: el código actual asume rutas `/api/tables`, `/api/menu/drinks` y `/api/orders` con header `X-API-Key`. Es muy probable que las reales sean distintas. Este paso se hace en la semana 1 para no descubrirlo en la semana 6.

Verificar: `docs/POS_REAL.md` completo y una llamada de prueba (por ejemplo, listar mesas) ejecutada con éxito contra el ambiente de pruebas del POS.**Reserva de la fase (imprevistos)****13 h**

Tiempo reservado para problemas de entorno (Docker en Windows, permisos, red). Si no se usa, se adelanta el Paso 1.1.

Fase 0 terminada cuando: ☐ repositorio remoto con rama `main` protegida · ☐ `.env` fuera de git · ☐ `docker compose up` con 6 servicios sanos · ☐ CI en verde · ☐ `docs/POS_REAL.md` lleno.

5. FASE 1 — Base de datos unificada y núcleo de la API

Semanas 2–3 · 80 horas. Objetivo: un solo esquema con migraciones, la API refactorizada por módulos, reservaciones montadas, contrato OpenAPI y tests de integración.

Paso 1.1 — Diseñar el esquema único

12 h

Qué hacer: Escribir `server/migrations/001_init.sql` unificando los tres SQL actuales. Cada tabla con `id` (uuid o bigserial), `created_at`, `updated_at`, y `nightclub_id` donde aplique (multi-club desde el inicio).

Grupo	Tablas
Núcleo	<code>nightclubs</code> , <code>users</code> (rol: guest, bartender, waiter, dancer, dj, light_tech, valet, hostess, manager, admin), <code>refresh_tokens</code> , <code>audit_log</code>
Mesas y pista	<code>tables</code> (x, y, radio, sección, capacidad, tipo, estado), <code>table_occupants</code> , <code>table_layouts</code>
Bebidas y pedidos	<code>drinks</code> , <code>inventory</code> , <code>drink_orders</code> , <code>drink_order_items</code>
Flirt	<code>flirts</code> , <code>flirt_reactions</code> , <code>user_blocks</code> , <code>user_reports</code> , <code>user_preferences</code> (opt-in)
Reservaciones	<code>reservations</code> , <code>reservation_payments</code> , <code>reservation_addons</code> , <code>reservation_rules</code> , <code>reservation_discounts</code> , <code>payment_methods</code>
Empleados	<code>employees</code> , <code>employee_bank_accounts</code> (cifrado), <code>exchange_rates</code> (con histórico), <code>withdrawals</code>
Propinas	<code>tips</code> , <code>song_requests</code> , <code>staff_drinks</code>
Salida segura	<code>taxi_rides</code> , <code>conduct_codes</code> , <code>emergency_contacts</code>
Valet	<code>parking_spots</code> , <code>valet_vehicles</code> , <code>valet_tickets</code>
Precios y POS	<code>pricing_rules</code> , <code>pos_integrations</code> , <code>pos_sync_log</code>
Finanzas	<code>transactions</code> (libro único: tipo, monto, moneda, origen, destino, referencia externa, estado) — solo inserción, nunca update/delete

Verificar: revisión del esquema por una segunda persona (o por el responsable del proyecto); el SQL corre limpio en una BD vacía.

Paso 1.2 — Runner de migraciones y seeds

6 h

Qué hacer: Crear `server/src/db/migrate.js` que aplique en orden los archivos de `migrations/` y registre los aplicados en la tabla `schema_migrations` (o usar `node-pg-migrate`). Crear `seeds/run.js` con: 1 club, 20 mesas con coordenadas, 30 bebidas, 10 usuarios (1 por rol), reglas de reservación y precios de ejemplo.

```
npm run migrate
npm run seed
psql $DATABASE_URL -c "\dt"
```

Verificar: ejecutar `npm run migrate` dos veces no falla ni duplica; `\dt` lista todas las tablas; los seeds cargan sin error.

Paso 1.3 — Refactorizar la API por módulos

20 h

Qué hacer: Dividir `src/index.js` en `routes/auth.js`, `routes/nightclubs.js`, `routes/drinks.js`, `routes/orders.js`, `routes/tables.js`, `routes/dashboard.js`, y `middleware` común.

- `middleware/auth.js`: JWT de acceso (15 min) + refresh token (30 días, en tabla, revocable). Función `requireRole('bartender', 'manager')`.
- `middleware/validate.js`: esquemas `zod` por ruta; error 400 con detalle.
- `middleware/errors.js`: manejador único; nunca exponer stack en producción.
- `helmet`, `express-rate-limit` (100 req/min por IP en auth), `pino-http` para logs JSON.
- Mover el `SoftRestaurant11Client` a `services/pos/softrestaurant11.js` (se ajustará en Fase 4).

Verificar: todas las rutas que existían antes siguen respondiendo igual (probar con la colección Postman/Bruno creada en este paso y guardada en `docs/api-collection.json`).

Paso 1.4 — Montar reservaciones

6 h

Qué hacer: En `src/index.js`: `app.use('/api', require('./routes/reservations'))`. Reemplazar `stripe.charges.create` por `PaymentIntents` (la API de charges está obsoleta). Dejar el cobro real para la Fase 3; en esta fase solo debe crear la reservación y el intent en modo test.

Verificar: `GET /api/nightclubs/1/reservations/availability?date=...` devuelve mesas del seed; `POST /api/nightclubs/1/reservations` crea fila en `reservations`.

Paso 1.5 — Contrato OpenAPI

10 h

Qué hacer: Escribir `server/openapi.yaml` (versión 3.1) con todas las rutas existentes y las que se crearán en la Fase 2 (aunque aún no existan, marcadas `x-status: planned`). Servir con `swagger-ui-express` en `/api/docs`.

Este archivo es el **acuerdo entre backend, web y móvil**. Si dos personas trabajan en paralelo, la de móvil empieza a generar su cliente desde aquí.

Verificar: `http://localhost:3000/api/docs` abre y permite ejecutar login desde la interfaz.

Paso 1.6 — Tests de integración

14 h

Qué hacer: Configurar `jest` + `supertest` con una BD de prueba (`easyflirt_test`) que se migra y limpia antes de cada suite. Escribir tests para: registro/login/refresh, listar bebidas, crear pedido y cambiar estado, sentarse en mesa, crear reservación, reglas de rol (un guest no puede confirmar pedidos).

Verificar: `npm test` en verde local y en CI (agregar servicio Postgres en el workflow). Cobertura mínima 60% en `src/routes`.

Reserva de la fase

12 h

Ajustes de esquema descubiertos al escribir tests; corrección de bugs heredados.

Hito H1 — Fase 1 terminada cuando: ☐ un solo esquema con migraciones · ☐ API modular · ☐ reservaciones montadas · ☐ `/api/docs` publicado · ☐ tests en verde en CI.

6. FASE 2 — Persistencia de los módulos de negocio

Semanas 3–5 · 100 horas. Objetivo: cada clase "en memoria" se convierte en servicio + repositorio Postgres + rutas HTTP + tests. Se hace en el orden de la tabla (por dependencia y riesgo financiero).

Patrón para cada módulo: (a) crear repositorio en db/, (b) refactorizar la clase para recibir el repositorio, (c) crear rutas, (d) agregar al OpenAPI, (e) tests, (f) PR.

Paso 2.1 — Mesas y pista (layout)

14 h

Origen: `dance-floor-manager.js`, `nightclub-tables.js`.

Rutas: `GET /nightclubs/:id/tables` · `PUT /nightclubs/:id/tables/layout` (manager) · `POST /tables/:id/seat` · `POST /tables/:id/release` · `GET /nightclubs/:id/tables/stats`.

Reglas: las coordenadas x/y y radio se guardan en `tables`; el estado (libre, ocupada, reservada, bloqueada) cambia solo por API; cada cambio publica evento `table_updated` (se conecta a Redis en Fase 3, por ahora una función `events.publish()` vacía).

Verificar: reiniciar el contenedor de API y el layout sigue igual; tests de sentar/liberar y de capacidad excedida.

Paso 2.2 — Precios dinámicos

8 h

Origen: `pricing-system.js`.

Rutas: `GET /nightclubs/:id/pricing` · `PUT /nightclubs/:id/pricing` (manager) · `POST /pricing/quote` (recibe mesa, fecha, hora, personas; devuelve precio calculado).

Reglas: reglas por día de la semana, hora, sección y evento especial en `pricing_rules`. **El precio siempre se calcula en el servidor;** el cliente solo lo muestra.

Verificar: tests con 5 escenarios (viernes VIP, martes barra, evento, descuento, sin regla → precio base).

Paso 2.3 — Flirt

12 h

Origen: `flirt-system.js`.**Rutas:** `POST /flirts` · `GET /flirts/received` · `GET /flirts/sent` · `POST /flirts/:id/react` · `POST /users/:id/block` · `POST /users/:id/report` · `PUT /me/preferences` (recibir flirts: sí/no).**Reglas obligatorias:** solo se puede enviar flirt a usuarios con opt-in activo; bloqueo impide envío; límite de 20 flirts/hora por usuario; los reportes quedan en `user_reports` visibles para manager; verificación de mayoría de edad en el registro (fecha de nacimiento obligatoria, 18+).**Verificar:** tests: opt-out rechaza envío (403), bloqueo rechaza, límite por hora responde 429.**Paso 2.4 — Empleados y cuentas (MXN/USD)**

16 h

Origen: `employee-account-system.js`.**Rutas:** `POST /employees` (manager) · `POST /employees/login` · `GET /employees/me/dashboard` · `PUT /employees/me` · `POST /employees/me/bank-accounts` · `GET /employees/me/earnings?from&to` · `POST /employees/me/withdrawals` · `GET /admin/withdrawals` · `POST /admin/withdrawals/:id/approve` · `GET/PUT /admin/exchange-rate`.**Reglas:** CLABE (18 dígitos con dígito verificador) y cuentas US se guardan cifradas con `pgcrypto` y solo se muestran enmascaradas (`****1234`). El tipo de cambio se guarda en `exchange_rates` con fecha; nunca constante en código. Los retiros nacen en estado `pending` y solo el manager los aprueba (pago manual fuera del sistema en esta versión).**Verificar:** tests de validación de CLABE, de que el retiro no exceda el saldo, y de que un empleado no pueda ver ganancias de otro.**Paso 2.5 — Propinas, bebidas a staff y solicitudes de canción**

14 h

Origen: `tip-dancer-system.js`.**Rutas:** `GET /nightclubs/:id/staff?role=` · `POST /tips` · `POST /staff/:id/drinks` · `POST /staff-drinks/:id/confirm` · `POST /song-requests` · `GET /dj/:id/song-requests` · `POST /song-requests/:id/play` · `GET /staff/:id/earnings` · `GET /nightclubs/:id/leaderboard`.**Reglas:** cada propina inserta una fila en `tips` y otra en `transactions` en la misma transacción SQL. Idempotencia: el cliente envía `client_request_id` (uuid); si se repite, se devuelve el mismo resultado sin duplicar. En esta fase la propina queda en estado `pending_payment`; el cobro real se conecta en Fase 3.**Verificar:** test de idempotencia (dos POST iguales → una sola fila); test de que `transactions` no acepta UPDATE (trigger).

Paso 2.6 — Salida segura / taxi

10 h

Origen: `taxi-system.js`.**Rutas:** `POST /taxi/requests` · `POST /taxi/requests/:id/confirm (staff)` · `GET /taxi/requests/:id/code` · `POST /taxi/codes/validate` · `POST /taxi/requests/:id/complete` · `GET /me/rides` · `GET /nightclubs/:id/emergency-contacts`.**Reglas:** código de conducta de 6 caracteres, expira a las 12 horas; contactos de emergencia por club (tabla, no constantes); se registra quién confirmó y a qué hora.**Verificar:** test de expiración y de validación de código inválido.**Paso 2.7 — Valet**

10 h

Origen: `valet-parking-system.js`.**Rutas:** `POST /valet/checkin` · `GET /valet/tickets/:id` · `POST /valet/tickets/:id/request (cliente)` · `POST /valet/tickets/:id/ready (valet)` · `POST /valet/tickets/:id/checkout` · `GET /valet/dashboard` · `GET /valet/report?date=`.**Reglas:** el ticket genera un código QR (contenido = id del ticket firmado); el cliente lo recibe en la app; al marcar "listo" se publica `vehicle_ready`.**Verificar:** flujo completo por tests; el mismo lugar de estacionamiento no puede asignarse dos veces.**Paso 2.8 — Registro de integraciones POS**

6 h

Origen: `pos-integration-system.js`.**Rutas:** `GET /pos/supported` · `POST /nightclubs/:id/pos` · `POST /nightclubs/:id/pos/test` · `GET /nightclubs/:id/pos/status`.**Reglas:** credenciales del POS cifradas en `pos_integrations`; el endpoint de prueba hace una llamada real al POS configurado. La lógica de sincronización real se construye en Fase 4.**Verificar:** guardar la integración de SoftRestaurant11 con los datos de `docs/POS_REAL.md` y que `/pos/test` responda OK.**Reserva de la fase**

10 h

Ajustes cruzados entre módulos (por ejemplo, propinas que dependen de empleados).

Fase 2 terminada cuando: ☐ ninguna clase guarda estado en memoria · ☐ reiniciar contenedores no pierde datos · ☐ cada módulo tiene rutas, OpenAPI y tests · ☐ cobertura $\geq 60\%$.

7. FASE 3 — Tiempo real y pagos

Semanas 5–7 · 90 horas. Objetivo: eventos en tiempo real fiables entre API y clientes, y cobros reales en modo sandbox con Stripe y Mercado Pago.

7.1 Tiempo real

Paso 3.1 — WS server autenticado y sin estado

10 h

Qué hacer: Reescribir `src/ws.js`: al conectar, el cliente envía el JWT en el primer mensaje (o en query `?token=`); el servidor valida y guarda `userId`, `role`, `nightclubId`. Eliminar el objeto `state` en memoria; el snapshot inicial se obtiene llamando a la API interna (`GET /nightclubs/:id/tables`).

Verificar: una conexión sin token válido se cierra con código 4401; con token recibe snapshot.

Paso 3.2 — Redis pub/sub entre API y WS

8 h

Qué hacer: `services/events.js` en la API publica en canal `ev2:events` mensajes `{type, nightclubId, audience:{roles?, userIds?, tableIds?}, payload}`. El WS se suscribe y reenvía solo a los clientes cuya audiencia coincide.

Eventos: `order_created`, `order_ready`, `order_delivered`, `table_updated`, `flirt_received`, `tip_received`, `song_requested`, `taxi_confirmed`, `vehicle_ready`, `reservation_paid`.

Verificar: con dos pestañas (cliente y bartender) un pedido creado aparece en la del bartender en menos de 1 segundo. Medir con `console.time`.

Paso 3.3 — Reconexión y entrega garantizada básica

6 h

Qué hacer: El cliente guarda `lastEventId`; al reconectar pide `GET /events/since/:id` (la API guarda los últimos 500 eventos por club en Redis, TTL 1 h). Ping/pong cada 25 s.

Verificar: apagar Wi-Fi del teléfono 30 s, encender: la app recupera los eventos perdidos sin recargar.

7.2 Pagos

Antes de empezar: crear cuentas de Stripe y Mercado Pago del negocio (no personales), obtener llaves de **prueba**, y guardar todo en el gestor de contraseñas. Las llaves productivas no se tocan hasta el Paso 7.12.

Paso 3.4 — Stripe: PaymentIntents + webhooks

14 h

Qué hacer: `services/payments/stripe.js`: `createIntent(amount, currency, metadata)`; ruta `POST /payments/stripe/intent`; webhook `POST /webhooks/stripe` con verificación de firma que marca `transactions` como `paid` y dispara la acción de negocio (confirmar reservación, acreditar propina). Apple Pay y Google Pay se habilitan desde el mismo intent con Payment Request Button.

Verificar: con tarjeta de prueba `4242 4242 4242 4242` una propina pasa a `paid` y el staff ve el evento `tip_received`. Usar `stripe listen --forward-to localhost:3000/webhooks/stripe` en local.

Paso 3.5 — Mercado Pago: Checkout Pro + OXXO/SPEI

14 h

Qué hacer: `services/payments/mercadopago.js`: crear preferencia con `external_reference = id` de `transactions`; ruta `POST /payments/mp/preference`; webhook `POST /webhooks/mp` que consulta el pago por id y actualiza estado. OXXO y SPEI quedan en `pending` hasta que el webhook confirme (puede tardar horas; la reservación se mantiene "en espera de pago" con límite de 24 h).

Verificar: pago de prueba con tarjeta de sandbox de MP aprobado → reservación confirmada; pago OXXO de prueba → reservación en espera.

Paso 3.6 — Pagos manuales (Zelle, Cash App, depósito)

6 h

Qué hacer: Estado `pending_manual` con referencia que el cliente escribe; ruta `POST /admin/payments/:id/confirm` para manager con nota y quién confirmó (queda en `audit_log`).

Verificar: flujo completo por tests; un usuario sin rol manager recibe 403.

Paso 3.7 — Reservación: depósito, cancelación y reembolso

10 h

Qué hacer: Conectar `routes/reservations.js` a los servicios de pago: el depósito (30% por defecto, tomado de `reservation_rules`) se cobra con intent; la cancelación calcula reembolso según la política (100% > 48 h, 50% 24–48 h, 0% < 24 h, configurable) y ejecuta `refund` en Stripe o MP; todo queda en `transactions`.

Verificar: tests de las tres ventanas de cancelación; reembolso visible en el dashboard de prueba de Stripe.

Paso 3.8 — Conciliación

6 h

Qué hacer: Ruta `GET /admin/reports/daily?date=` que suma `transactions` por tipo y proveedor, y job nocturno que compara con la lista de pagos de Stripe/MP del día y marca diferencias.

Verificar: el reporte del día de pruebas cuadra con los dashboards de Stripe y MP.

Reserva de la fase

16 h

Los webhooks y los casos borde de pago (doble clic, pago parcial, timeout) suelen consumir más de lo previsto.

Hito H2 — Fase 3 terminada cuando: ☐ WS autenticado sin estado · ☐ evento en <1 s entre pestañas · ☐ pago sandbox Stripe y MP extremo a extremo · ☐ reembolso funcional · ☐ reporte diario cuadra.

8. FASE 4 — Integración real con SoftRestaurant11

Semanas 6–8 · 70 horas. Objetivo: menú, inventario, comandas y estados sincronizados con el POS real.
Regla absoluta: todo se prueba contra la base/sucursal de pruebas del POS. Nunca contra la caja activa del club hasta el piloto de la Fase 7.

Paso 4.1 — Ajustar el cliente al mecanismo real

12 h

Qué hacer: Con `docs/POS-REAL.md` (Paso 0.9), reescribir `services/pos/softrestaurant11.js` para el mecanismo real. Tres escenarios posibles:

- **API REST:** ajustar URLs, autenticación y mapeo de campos.
- **Conexión a SQL Server del POS:** instalar `mssql`, usuario de solo lectura para catálogo/inventario y procedimiento almacenado o tabla intermedia para comandas (acordar con el proveedor).
- **Sin API (solo exportación):** importación programada de archivos + comandas manuales; documentar la limitación y proponer al proveedor la alternativa.

Verificar: `POST /nightclubs/:id/pos/test` devuelve el número real de mesas y productos del POS de prueba.

Paso 4.2 — Simulador pos-mock

6 h

Qué hacer: Pequeño servidor Express en `server/tools/pos-mock/` que imita el mecanismo real (mismas rutas/tablas) con datos fijos. Se agrega a `docker-compose.yml` con perfil `dev`. Los tests y el entorno local lo usan; solo staging/producción hablan con el POS real.

Verificar: `npm test` pasa sin acceso al POS real.

Paso 4.3 — Sincronización de catálogo (menú y precios)

10 h

Qué hacer: Job cada 15 min (`node-cron`) + ruta `POST /nightclubs/:id/pos/sync/menu`. Mapeo POS → `drinks` por `pos_product_id`; productos que desaparecen del POS se marcan `inactive`, no se borran. Registrar cada corrida en `pos_sync_log`.

Verificar: cambiar un precio en el POS de prueba y verlo en la app en ≤15 min o al forzar sync.

Paso 4.4 — Sincronización de inventario

8 h

Qué hacer: Job cada 5 min que actualiza `inventory.quantity`; la creación de pedido rechaza (409) si no hay stock y el WS emite `drink_unavailable` para ocultar el producto en las apps.

Verificar: poner un producto en 0 en el POS → el pedido desde la app es rechazado con mensaje claro.

Paso 4.5 — Envío de comandas al POS

14 h

Qué hacer: Al crear un pedido en la app: (1) se guarda en `drink_orders` con estado `sending_to_pos`, (2) se envía al POS con la mesa correcta (mapeo `tables.pos_table_id`), (3) se guarda `pos_order_id` y estado `sent`. Si el POS falla, el pedido queda `pos_error`, se reintenta 3 veces con espera creciente y el bartender ve una alerta para capturarlo a mano.

Verificar: un pedido desde la web aparece en la pantalla del POS de prueba con la mesa y los productos correctos. Apagar el pos-mock y comprobar el flujo de error y reintento.

Paso 4.6 — Estados de vuelta (webhook o polling)

8 h

Qué hacer: Si el POS tiene webhooks: endpoint `POST /webhooks/pos` con verificación de firma. Si no: polling cada 30 s de los pedidos abiertos. Estados `preparing` → `ready` → `delivered` → `paid` actualizan `drink_orders` y publican eventos WS.

Verificar: cerrar una cuenta en el POS de prueba → la mesa se libera en la app.

Paso 4.7 — Conciliación diaria app vs POS

6 h

Qué hacer: Reporte `GET /admin/reports/pos-reconciliation?date=`: pedidos de la app, pedidos con `pos_order_id`, pedidos en error, diferencia de importes.

Verificar: tras un día de pruebas, el reporte muestra 0 pedidos sin `pos_order_id` o explica cada uno.

Reserva de la fase

6 h

Diferencias entre documentación del proveedor y comportamiento real.

Hito H3 — Fase 4 terminada cuando: ☐ menú e inventario vienen del POS · ☐ comanda de la app aparece en el POS de prueba · ☐ estados regresan a la app · ☐ reporte de conciliación en 0 diferencias.

9. FASE 5 — Conectar las apps web a la API

Semanas 7–9 · 90 horas. Objetivo: todas las pantallas HTML funcionan con datos del servidor, con roles y en teléfono. Se conserva HTML + Tailwind sin build.

Paso 5.1 — Módulo común web/js/api.js

10 h

Qué hacer: Un solo archivo con: `login()`, `logout()`, guardado de tokens (access en memoria, refresh en `localStorage`), `apiFetch(path, opts)` que agrega el token y renueva si expira, `connectWS(onEvent)` con reconexión y `lastEventId`, `t(key)` para textos ES/EN, `money(amount, currency)` para MXN/USD, y `config.js` con `API_URL` / `WS_URL` por entorno.

Verificar: página de prueba `web/dev-api-test.html` que hace login, lista bebidas y recibe un evento WS.

Paso 5.2 — Pantalla principal del cliente (index.html)

16 h

Qué hacer: Reemplazar datos simulados de `index-ev2-branded.html`: registro/login con fecha de nacimiento, mapa de mesas real (coordenadas de la BD), sentarse, menú desde el POS, crear pedido, enviar bebida a otra mesa, enviar flirt (con opt-in), actividad en vivo por WS.

Verificar: flujo completo en teléfono contra el entorno local; sin ningún dato hardcodeado (buscar "mock", "demo", arrays fijos).

Paso 5.3 — Selector de mesas y reservación (table-selector.html)

10 h

Qué hacer: Disponibilidad por fecha/hora, cotización de precio desde `/pricing/quote`, extras, código de descuento, pago del depósito con Stripe Elements o redirección a Mercado Pago, confirmación y "mis reservaciones".

Verificar: reservación con pago sandbox termina en estado `confirmed`.

Paso 5.4 — Propinas y canciones (tip-dancers-system.html)

10 h

Qué hacer: Lista real de staff por rol, propina con monto rápido o libre, pago con tarjeta guardada / Apple Pay / Google Pay / MP, solicitud de canción al DJ con propina, tablero de las más pedidas.

Verificar: el DJ (en otra pestaña con su usuario) ve la solicitud en vivo.

Paso 5.5 — Portal de empleados (employee-portal.html)

10 h

Qué hacer: Login de empleado, dashboard de ganancias con toggle MXN/USD, historial, registro de cuenta bancaria (formulario CLABE/US con validación), solicitud de retiro, estado de retiros. Vista de manager: aprobar retiros, editar tipo de cambio, ver reportes.

Verificar: un retiro solicitado por la bailarina aparece en la vista del manager y al aprobarlo cambia de estado en la del empleado.

Paso 5.6 — Salida segura (safe-departure.html)

6 h

Qué hacer: Solicitud de taxi, aceptación del código de conducta, pantalla del código en grande, contactos de emergencia del club, historial.

Verificar: el staff confirma desde su pantalla y el cliente recibe `taxi_confirmed` con el código.

Paso 5.7 — Pantallas de staff nuevas: bartender.html, staff.html, valet.html, manager.html

16 h

Qué hacer:

- **bartender.html:** cola de pedidos en vivo, botones grandes "en preparación / listo / entregado", alerta de `pos_error`.
- **staff.html** (mesero, bailarina, DJ, técnico de luces): mis mesas, propinas recibidas, bebidas enviadas para confirmar, solicitudes de canción (DJ).
- **valet.html:** check-in con placa y foto, lista de vehículos, "listo", check-out, dashboard.
- **manager.html:** reutilizar `pricing-dance-floor-flirt.html` como base: editor de layout (arrastrar mesas y guardar), precios, reportes diarios, conciliación POS, reportes de usuarios, aprobación de pagos manuales.

Verificar: cada rol al iniciar sesión es redirigido a su pantalla; un rol no puede abrir la de otro (403 en API y redirección en web).

Paso 5.8 — PWA instalable

4 h

Qué hacer: `manifest.json` (nombre, íconos desde `web/images`, color de tema), service worker que cachea los archivos estáticos (no las respuestas de API), pantalla de "sin conexión".

Verificar: en Android Chrome aparece "Agregar a pantalla de inicio" y la app abre a pantalla completa.

Paso 5.9 — Prueba de usabilidad en contexto de club**4 h**

Qué hacer: Con 3 personas que no conozcan el sistema, en un cuarto con poca luz y usando el teléfono con una mano: pedir una bebida, enviar una bebida, pedir taxi. Anotar cada duda o error. Corregir tamaños de botón, contraste y textos.

Verificar: las 3 personas completan las 3 tareas sin ayuda.

Reserva de la fase**4 h**

Ajustes visuales tras la prueba de usabilidad.

Hito H4 — Fase 5 terminada cuando: ☐ flujo de cliente completo con datos del servidor · ☐ flujo de staff completo · ☐ roles protegidos · ☐ PWA instalable · ☐ prueba de usabilidad aprobada.

10. FASE 6 — Apps nativas iOS y Android

Semanas 8–12 · 150 horas. Objetivo: apps compilables, conectadas a staging, en TestFlight y prueba interna de Play. Empieza cuando la API es estable (fin de Fase 3). Si solo hay una persona, esta fase puede posponerse: la PWA de la Fase 5 permite operar el club mientras tanto.

Requisitos previos: Mac con Xcode 16+ y cuenta Apple Developer (99 USD/año, la aprobación tarda hasta 1 semana: solicitarla en la semana 1). Cuenta Google Play Console (25 USD única vez). Cuenta Stripe con Apple Pay y Google Pay habilitados.

10.1 iOS (75 h)

Paso 6.1 — Proyecto Xcode y compilación

12 h

Qué hacer: Crear `mobile/ios/EV2.xcodeproj` (SwiftUI, iOS 16+, bundle `com.ev2clandestinoz.app`). Incorporar los 11 archivos `.swift`, resolver duplicados (`EasyFlirtView` vs `EV2BrandedApp`), agregar assets (logo, colores de marca), `Info.plist` con permisos (cámara para QR de valet, notificaciones).

Verificar: compila y corre en simulador mostrando la pantalla de login.

Paso 6.2 — Cliente de API generado y configuración por entorno

10 h

Qué hacer: Generar cliente con `openapi-generator` (swift5) desde `server/openapi.yaml` o escribir un `APIClient` con `async/await` a mano si el generado es pesado. Reemplazar URLs fijas de `NetworkService.swift` por `Config.plist` por esquema (Debug=local, Staging, Release). Token en Keychain.

Verificar: login y lista de mesas contra staging.

Paso 6.3 — Pantallas principales

28 h

Qué hacer: Mapa de mesas, menú y pedido, enviar bebida/flirt, reservación, propinas y canciones, salida segura, valet (escaneo QR), perfil. Reutilizar `WebSocketManager.swift` con auth por token y reconexión.

Verificar: mismo flujo de cliente de la Fase 5 completado en dispositivo real.

Paso 6.4 — Apple Pay y notificaciones push

15 h

Qué hacer: Merchant ID, certificado de Apple Pay en Stripe, `PKPaymentRequest` para propinas y depósitos. APNs: registrar token en `POST /me/devices`; la API envía push para `order_ready`, `taxi_confirmed`, `vehicle_ready`, `tip_received`.

Verificar: propina con Apple Pay sandbox aprobada; push recibida con la app cerrada.

Paso 6.5 — TestFlight

10 h

Qué hacer: Íconos, splash, capturas, política de privacidad (URL pública), build firmado, subida a App Store Connect, grupo interno de testers (staff del club).

Verificar: 5 personas instalan desde TestFlight y completan el flujo.

10.2 Android (75 h)

Paso 6.6 — Proyecto Android Studio y compilación

12 h

Qué hacer: Crear `mobile/android/` (Kotlin, Compose, minSdk 26, paquete `com.ev2clandestinoz.app`). Incorporar `EV2Android.kt` y `ReservationSystem.kt`, agregar navegación (Navigation Compose), Retrofit + Moshi, OkHttp WebSocket, `DataStore/EncryptedSharedPreferences`, `AndroidManifest.xml` con permisos.

Verificar: compila y corre en emulador con login.

Paso 6.7 — Cliente de API y configuración por build type

8 h

Qué hacer: Interfaces Retrofit según OpenAPI; `BuildConfig.API_URL` por `debug/staging/release`; interceptor de refresh de token.

Verificar: login y mesas contra staging.

Paso 6.8 — Pantallas principales

28 h

Qué hacer: Misma lista de pantallas que iOS (Paso 6.3), con Material 3 y la paleta EV2.

Verificar: flujo de cliente completo en dispositivo real.

Paso 6.9 — Google Pay y FCM

15 h

Qué hacer: Google Pay vía Stripe (Payment Sheet), FCM con token registrado en `POST /me/devices`, canal de notificaciones.

Verificar: propina con Google Pay en test aprobada; push recibida con la app cerrada.

Paso 6.10 — Prueba interna en Play Console

12 h

Qué hacer: Íconos adaptativos, capturas, política de privacidad, cuestionario de seguridad de datos, AAB firmado, pista de prueba interna con el staff.

Verificar: 5 personas instalan desde el enlace de prueba interna y completan el flujo.

Hito H6 — Fase 6 terminada cuando: ☐ ambas apps compilan en CI · ☐ flujo de cliente completo contra staging · ☐ pagos con wallet en sandbox · ☐ push funcionando · ☐ TestFlight y prueba interna activos.

11. FASE 7 — Servidor, staging y datos reales

Semanas 9–11 · 80 horas. Objetivo: servidor endurecido, despliegue automático a staging, datos reales del club cargados, dos pilotos completados. Esta fase corre en paralelo con la Fase 6.

11.1 Servidor (34 h)

Paso 7.1 — Contratar VPS y dominio

2 h

Qué hacer: VPS Ubuntu 24.04, 4 vCPU / 8 GB RAM / 80 GB SSD (por ejemplo Hetzner CPX31 o DigitalOcean 8 GB; 20–30 USD/mes). Dominio propio; registros DNS A: `app.`, `api.`, `staging.`, `staging-api.` apuntando al VPS.

Verificar: `ping app.tudominio.com` responde la IP del VPS.

Paso 7.2 — Endurecer el servidor

4 h

```
adduser deploy && usermod -aG sudo deploy
# copiar llave SSH a /home/deploy/.ssh/authorized_keys
sed -i 's/^#?PasswordAuthentication.*/PasswordAuthentication no/; s/^#?
PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config
systemctl restart ssh
ufw allow 22 && ufw allow 80 && ufw allow 443 && ufw enable
apt install -y fail2ban unattended-upgrades
dpkg-reconfigure -pnow unattended-upgrades
```

Verificar: entrar solo con llave como `deploy`; `ssh root@` rechazado; `ufw status` muestra 3 reglas.

Paso 7.3 — Docker y compose de producción

6 h

Qué hacer: Instalar Docker Engine + Compose plugin. Crear `deploy/docker-compose.prod.yml`: Postgres y Redis **sin puertos publicados**, `restart: always`, límites de memoria, `logging: json-file, max-size 10m, max-file 5`, imágenes desde GHCR con tag por commit. Un stack para staging (`-p ev2-staging`) y otro para producción (`-p ev2-prod`), cada uno con su `.env` en `/srv/ev2/<env>/.env` con permisos 600.

Verificar: `docker compose -p ev2-staging up -d` levanta el stack; `ss -tlnp` muestra solo 22, 80, 443 abiertos al exterior.

Paso 7.4 — Reverse proxy y SSL

4 h

Qué hacer: Usar **Caddy** en lugar de nginx para que el SSL de Let's Encrypt sea automático (menos mantenimiento). `deploy/Caddyfile` con un bloque por subdominio: `app.` → `web`, `api.` → `api:3000` y `/ws` → `ws:4000`, y lo mismo para `staging-*`. Si se prefiere mantener nginx, usar el `nginx-nightclub.conf` existente + `certbot`.

Verificar: `https://staging-api.tudominio.com/health` responde 200 con candado válido; calificación A en `ssllabs.com`.

Paso 7.5 — Respaldos

5 h

Qué hacer: Script `deploy/scripts/backup.sh`: `pg_dump` comprimido cada noche a las 6:00 (después del cierre del club), subido a Backblaze B2 o S3 con `rclone`, retención 30 días. Cron en el usuario `deploy`. Script `restore.sh` documentado.

Verificar: restaurar el respaldo en el stack de staging y comprobar que los datos están. Repetir esta prueba una vez al mes (anotar en calendario).

Paso 7.6 — Monitoreo y alertas

5 h

Qué hacer: Uptime Kuma (contenedor propio) vigilando `/health` de API y WS de staging y producción cada 60 s, con alertas por Telegram o correo. Alerta de disco > 80% y de contenedor reiniciado. Logs con `docker logs` y rotación (ya configurada en 7.3); opcional Loki + Grafana más adelante.

Verificar: detener el contenedor de API y recibir la alerta en menos de 2 minutos.

Paso 7.7 — CI/CD: despliegue automático

8 h

Qué hacer: Extender `.github/workflows/`: en `main`, construir imágenes → subir a GHCR → por SSH ejecutar `deploy/scripts/deploy.sh staging <tag>` (`pull`, `npm run migrate`, `up -d`, `health check`, y si falla, volver al tag anterior). Despliegue a producción con el mismo script pero con **aprobación manual** en GitHub Environments. Escribir `docs/RUNBOOK.md`: cómo desplegar, cómo hacer rollback, cómo restaurar respaldo, a quién avisar.

Verificar: un merge a `main` aparece en staging en menos de 10 minutos sin intervención; un rollback manual funciona.

11.2 Datos reales y pilotos (46 h)

Paso 7.8 — Carga de datos reales del club en staging

10 h

Qué hacer: (1) Club real en `nightclubs` con nombre, dirección, capacidad, contactos de emergencia. (2) Layout real: medir o fotografiar el piso y dibujar las mesas con el editor de manager; asignar `pos_table_id` a cada mesa. (3) Menú y precios desde el POS real (sync). (4) Staff real con rol y correo; contraseña temporal que deben cambiar. (5) Reglas de reservación y precios reales revisados con el dueño. (6) Usuarios de prueba para el piloto.

Los datos personales reales (staff, cuentas bancarias) se cargan en staging solo si staging tiene el mismo nivel de seguridad que producción (SSL, sin puertos abiertos, respaldos). Si no, usar datos ficticios en staging y cargar los reales directamente en producción en el Paso 7.12.

Verificar: el dueño o manager revisa el mapa, el menú y la lista de staff en staging y firma su conformidad.

Paso 7.9 — Preparación del piloto 1

6 h

Qué hacer: Elegir una noche de afluencia media. Alcance: **una sección** del club, 3–5 meseros, 1 bartender, 1 manager; pagos **solo en sandbox o efectivo/POS** (sin cobros reales). Preparar: Wi-Fi dedicado para staff, teléfonos cargados con la PWA instalada, hoja impresa de "qué hacer si falla" (ver sección 13), canal de WhatsApp de soporte, y capacitación de 45 minutos al staff ese mismo día.

Verificar: lista de asistencia de la capacitación; cada teléfono abre la app y hace login.

Paso 7.10 — Piloto 1 (una noche)

8 h

Qué hacer: El desarrollador está presente toda la noche observando. Anotar: cada error, cada pedido que no llegó al POS, latencia percibida, dudas del staff, caídas de red. Al cierre, exportar `pos_sync_log`, reporte de conciliación y logs.

Verificar: informe de piloto en `docs/pilotos/piloto1.md` con lista de incidencias clasificadas (bloqueante / importante / menor).

Paso 7.11 — Corrección post-piloto 1

12 h

Qué hacer: Resolver todas las incidencias bloqueantes e importantes; desplegar a staging; repetir las pruebas afectadas.

Verificar: 0 bloqueantes abiertos.

Paso 7.12 — Corte a producción y piloto 2 (pagos reales pequeños)**10 h**

Qué hacer: (1) Crear `.env` de producción: `JWT_SECRET` nuevo de 64 caracteres, llaves **live** de Stripe y MP, `ALLOWED_ORIGINS` real, credenciales del POS de producción. (2) Desplegar producción con aprobación manual. (3) Cargar datos reales (si no se hizo en staging). (4) Piloto 2: misma sección, ahora con propinas y depósitos reales de montos pequeños (por ejemplo, máximo 200 MXN) durante una noche. (5) Al día siguiente, conciliar con Stripe, MP y POS.

Verificar: reporte de conciliación con 0 diferencias; informe `docs/pilotos/piloto2.md`.

Hito H5 — Fase 7 terminada cuando: ☐ staging y producción por HTTPS · ☐ respaldo restaurado con éxito · ☐ alertas probadas · ☐ despliegue automático a staging · ☐ piloto 1 y 2 completados con informes · ☐ `docs/RUNBOOK.md` escrito.

12. FASE 8 — Seguridad, cumplimiento y cierre

Semanas 11–13 · 60 horas. Objetivo: revisión de seguridad, obligaciones legales, manuales por rol, capacitación y plan de soporte. Termina con la apertura completa a todo el club.

Paso 8.1 — Revisión de seguridad de la API

12 h

Qué hacer: Recorrer la lista OWASP Top 10 sobre cada módulo: inyección (todas las consultas parametrizadas), autenticación (expiración, revocación de refresh), control de acceso (probar cada ruta con cada rol), exposición de datos (respuestas sin campos sensibles), rate-limit en auth, pagos y flirts, CORS estricto a los dominios reales, cabeceras de `helmet`. Ejecutar `npm audit` y resolver críticos. Ejecutar OWASP ZAP en modo baseline contra staging.

Verificar: lista de hallazgos en `docs/SEGURIDAD.md` con estado; 0 críticos abiertos.

Paso 8.2 — Datos personales y consentimiento

8 h

Qué hacer: Aviso de privacidad conforme a la LFPDPPP (México) redactado o revisado por un abogado; pantalla de aceptación en registro; consentimiento explícito e independiente para flirt y para ubicación; ruta `DELETE /me` que anonimiza la cuenta; política de retención (códigos de conducta y viajes de taxi: 90 días; logs: 30 días).

Verificar: el aviso está publicado en una URL pública (necesaria también para App Store y Play); la eliminación de cuenta funciona.

Paso 8.3 — Datos financieros

6 h

Qué hacer: Confirmar que nunca se guarda número de tarjeta (solo tokens de Stripe/MP); cuentas bancarias cifradas y enmascaradas; `transactions` con trigger que impide UPDATE/DELETE; `audit_log` para aprobaciones de retiros y pagos manuales; acceso a reportes financieros solo para manager/admin.

Verificar: consulta directa a la BD muestra cuentas cifradas; intento de UPDATE en `transactions` falla.

Paso 8.4 — Términos de uso y código de conducta

4 h

Qué hacer: Publicar términos de uso y el código de conducta (base en `docs/SAFE_DEPARTURE_IMPLEMENTATION.md`); aceptación obligatoria en el registro con fecha y versión guardadas.

Verificar: la tabla `users` guarda `terms_version` y `terms_accepted_at`.

Paso 8.5 — Manuales de una hoja por rol

10 h

Qué hacer: En `docs/operacion/`, un PDF de una página, con capturas, para: cliente (cómo instalar y pedir), mesero, bartender, bailarina/DJ/técnico, valet, manager (incluye qué hacer si falla el POS o la red). Español, letra grande, listo para imprimir y pegar en la barra.

Verificar: una persona del staff sigue el manual de su rol sin preguntar nada.

Paso 8.6 — Capacitación y apertura completa

8 h

Qué hacer: Sesión de 1 hora por turno con todo el staff; entrega de manuales; ampliación a todas las secciones del club; anuncio a clientes (QR en mesas con enlace a la PWA / tiendas).

Verificar: primera noche completa con todas las secciones y reporte de conciliación en 0 diferencias.

Paso 8.7 — Plan de soporte (4 semanas)

12 h

Qué hacer: Desarrollador disponible por WhatsApp las noches de operación durante 4 semanas; revisión diaria de alertas, logs y conciliación (30 min); reunión semanal de 30 min con el manager para priorizar mejoras; lista de mejoras en el repositorio (issues).

Verificar: bitácora de soporte semanal en `docs/soporte/`.

Hito H7 — Producción: ☐ seguridad revisada · ☐ aviso de privacidad y términos publicados · ☐ manuales entregados · ☐ staff capacitado · ☐ apertura completa realizada · ☐ soporte activo.

13. Riesgos y qué hacer si ocurren

Riesgo	Impacto	Prevención	Si ocurre
La integración real de SoftRestaurant11 no es como asume el código	Alto	Paso 0.9 en la semana 1; <code>pos-mock</code> para no bloquear al resto.	Negociar con el proveedor acceso a SQL Server o exportaciones; ajustar Paso 4.1; mientras tanto el bartender captura a mano.
Stripe limitado en México / OXXO y SPEI	Alto	Mercado Pago como proveedor principal en MX desde el diseño.	Operar solo con MP y pagos manuales; Stripe queda para USD.
Retiros a cuentas de empleados (dinero real)	Alto	Primera versión solo registro + aprobación manual.	No automatizar hasta tener proveedor de payouts y contrato.
Apps nativas retrasan el lanzamiento	Medio	PWA en Fase 5 permite operar sin tiendas.	Lanzar con PWA; nativas llegan después sin bloquear.
Wi-Fi/datos débiles dentro del club	Medio	Reconexión automática, <code>lastEventId</code> , idempotencia, Wi-Fi dedicado para staff.	Repetidor Wi-Fi en la sección; el POS sigue siendo la fuente de verdad.
Documentación anterior promete más de lo que hay	Medio	Este manual reemplaza los resúmenes como fuente de verdad.	Remitir a la sección 1 de este manual.
Pérdida de datos / sin respaldos	Alto	Pasos 0.1 y 7.5; prueba de restauración mensual.	Ejecutar <code>restore.sh</code> según <code>docs/RUNBOOK.md</code> .
Flirt y datos sensibles (acoso, menores)	Alto	Opt-in, bloqueo, reporte, 18+, aviso de privacidad.	Desactivar el módulo flirt con la bandera <code>ENABLE_FLIRT=false</code> sin afectar el resto.
Cuenta Apple Developer tarda en aprobarse	Medio	Solicitarla en la semana 1.	Avanzar Android primero; iOS por PWA mientras tanto.
Un paso toma más del doble del tiempo estimado	Medio	Reservas por fase (≈10–15%).	Detener, reportar al responsable, reajustar calendario antes de continuar.

13.1 Hoja "qué hacer si falla" para la noche del piloto (imprimir para el manager)

Síntoma	Acción inmediata
La app no abre o no carga	Revisar Wi-Fi del teléfono → abrir <code>https://api.../health</code> → si no responde, avisar al desarrollador y seguir con el POS como siempre.
El pedido no llega al bartender	Bartender revisa alerta de <code>pos_error</code> en su pantalla → captura el pedido a mano en el POS → reportar número de pedido.

Un pago se quedó "en proceso"	No volver a cobrar. Manager revisa en <code>manager.html</code> → pagos pendientes; si persiste 10 min, cobrar por POS y anotar la referencia.
Un cliente reporta acoso por flirt	Manager bloquea al usuario desde <code>manager.html</code> → reportes; si es grave, desactivar el módulo (<code>ENABLE_FLIRT=false</code>) y avisar.
Se cayó la red del club	Todo el staff pasa al POS. Al volver la red, la app recupera eventos; conciliar al cierre.

14. Definición de "listo para producción"

El sistema se considera listo **solo cuando se cumplen todos** estos puntos:

	Criterio	Evidencia
☐	Todos los módulos guardan en Postgres y sobreviven a un reinicio.	Fase 2
☐	Tests automatizados en CI cubren auth, pedidos, reservaciones, propinas y pagos (≥ 60%).	CI en verde
☐	Pagos reales probados en montos pequeños con Stripe y Mercado Pago.	Piloto 2
☐	Integración con SoftRestaurant11 validada en piloto real.	Conciliación 0 diferencias
☐	HTTPS, respaldos diarios con restauración probada, monitoreo con alertas.	Fase 7
☐	Dos pilotos en el club completados con incidencias resueltas.	docs/pilotos/
☐	Staff capacitado y manuales entregados.	Lista de asistencia
☐	Aviso de privacidad, términos y código de conducta publicados y aceptados en registro.	URL pública
☐	Revisión de seguridad sin críticos abiertos.	docs/SEGURIDAD.md

15. Hoja de control de avance

Llenar al terminar cada fase. Entregar copia al responsable del proyecto.

Fase	Horas est.	Horas reales	Fecha inicio	Fecha fin	Firma desarrollador	Firma responsable
0	40					
1	80					
2	100					
3	90					
4	70					
5	90					
6	150					
7	80					
8	60					
Total	760					

Hitos

Hito	Descripción	Fecha prevista	Fecha real	Firma
H1	API unificada sobre Postgres, tests en verde	Fin semana 3		
H2	Pago sandbox extremo a extremo + tiempo real	Fin semana 7		
H3	Pedido de la app aparece en SoftRestaurant11 de prueba	Fin semana 8		
H4	Web completa conectada; staging en HTTPS	Fin semana 9		
H5	Piloto 1 en el club	Semana 10		
H6	Apps en TestFlight / prueba interna; piloto 2	Fin semana 12		
H7	Producción y apertura completa	Semana 13		

Notas e incidencias

Fecha	Paso	Descripción	Resuelto (fecha)